

BOOTCAMP MYHUB · SUPPORT DE COURS

Jour 2. Programmation Python.

Support de cours à lire, annoter et garder. Imprime-le ou suis-le à l'écran. Les corrigés se font ensemble, pendant la séance.

Gratuit

Jour 2 · 2 h

Débutant bienvenu

En visio · 12 h GMT

AUJOURD'HUI TU REPARS AVEC

Un premier script Python : variables, `print`, `input`, décisions `if`, et un fichier qui écrit le texte de ta vitrine.

Garde ce support et le dossier `vitrine` du jour 1.

Prépare ça, rien de plus

- ☐ Un ordinateur (le téléphone seul ne suffit pas pour coder).
- ☐ Le dossier `vitrine` du jour 1, avec `index.html`.
- ☐ Python installé, ou un onglet prêt : <https://www.online-python.com/>.
- ☐ Un éditeur : VS Code, Bloc-notes, ou l'éditeur du site en ligne.

PAS DE PYTHON SUR LA MACHINE ?

On code dans le navigateur sur <https://www.online-python.com/>. Même exercice, zéro install. À la fin, tu copies le code dans un fichier `bio.py` pour le déposer sur MyHub.

OBJECTIF DU JOUR

Faire parler l'ordinateur en Python, puis lui faire écrire le texte de ta vitrine. Pas besoin d'avoir déjà programmé.

Les trois temps du jour

	ON TRAVAILLE	TU REPARS AVEC
1	<code>print</code> , variables, types	Python parle, tu donnes un nom aux valeurs
2	<code>input</code> et <code>if</code>	Un script qui écoute et décide
3	Mini-projet bio	Ton fichier <code>bio.py</code> + le texte de la vitrine

1. Un programme, c'est une recette

Hier, tu as écrit du **HTML** : la structure d'une page. Aujourd'hui, tu écris des **instructions** : l'ordinateur les lit, dans l'ordre, et les exécute. C'est un **programme**.

Analogie cuisine : la recette n'est pas le plat. Python, c'est la langue de la recette. L'**interpréteur** Python, c'est le cuisinier qui lit et exécute, ligne par ligne.

Le fichier

On enregistre le programme dans un fichier qui finit par `.py`. Exemple : `bio.py`. C'est du texte, comme `index.html` hier.

L'exécution

Python lit le fichier du haut vers le bas. S'il bute sur une ligne, il s'arrête et affiche une **erreur**. Ce n'est pas toi qui es « nul » : c'est la recette qui est floue.

Le premier mot : `print`

`print` fait parler l'ordinateur. Il affiche quelque chose à l'écran. Le texte va entre guillemets.

```
print("Bonjour")
print("Je m'appelle Awa")
```

À l'écran, tu vois :

```
Bonjour
Je m'appelle Awa
```

- Les guillemets `"` ou `'` entourent un **texte**.
- Sans guillemets, Python cherche un **nom** (une variable).
- Chaque `print` passe à la ligne suivante.

PIÈGE DU SOIR

`print(Bonjour)` sans guillemets → erreur `NameError`. Python cherche une variable appelée `Bonjour`. Le texte, lui, vit entre guillemets.

2. Une variable, c'est une boîte avec un nom

Au lieu de réécrire « Awa » partout, tu ranges la valeur dans une **variable**. Tu colles une étiquette sur une boîte.

```
prenom = "Awa"
print(prenom)
print("Bonjour", prenom)
```

À l'écran :

```
Awa
Bonjour Awa
```

Le signe `=` n'est pas « égal » comme en maths. Il **range** : « mets "Awa" dans la boîte `prenom` ».

Règles pour nommer

OK	PAS OK	POURQUOI
<code>prenom</code>	<code>prénom</code>	Pas d'accent dans le nom.
<code>age</code>	<code>2age</code>	Ne commence pas par un chiffre.
<code>envie_1</code>	<code>envie 1</code>	Pas d'espace : utilise <code>_</code> .
<code>message</code>	<code>print</code>	N'écrase pas un mot de Python.

CONVENTION DU SOIR

Minuscules, mots collés avec un tiret bas : `prenom`, `phrase`, `envie_1`. Un nom dit ce qu'il y a dans la boîte.

Deux types pour commencer

Texte • `str`

Entre guillemets. `"Awa"`, `"20"`. On peut coller deux textes avec `+`.

Nombre entier • `int`

Sans guillemets. `20`, `12`. On peut additionner : `20 + 2` donne `22`.

```
prenom = "Awa"
age = 20
print(prenom + " a " + str(age) + " ans")
print(age + 1)
```

`str(age)` transforme le nombre en texte, pour pouvoir le coller. `age + 1` calcule : 21.

"20" N'EST PAS 20

"20" + "1" donne "201" (collage de textes). $20 + 1$ donne 21 (calcul). Les guillemets changent le métier de la valeur.

Exercices 1 • print et variables

Écris tes réponses au crayon, ou dans un fichier `exos.py`. On corrige ensemble.

1.1 • 4 MIN • QU'EST-CE QUI S'AFFICHE ?

Sans lancer le programme, prédis l'écran :

```
ville = "Lomé"  
print("J'habite à")  
print(ville)
```

1.2 • 4 MIN • RÉPARE

Chaque ligne a un problème. Corrige-les :

```
print(Bonjour)  
prenom = Awa  
print prenom  
lville = "Lomé"
```

1. Ligne 1 :

.....

2. Ligne 2 :

.....

3. Ligne 3 :

.....

4. Ligne 4 :

.....

1.3 • 4 MIN • TA CARTE D'IDENTITÉ

Écris 4 lignes : une variable `prenom`, une variable `ville`, puis deux `print` qui affichent « Je m'appelle ... » et « J'habite à ... ».

.....
.....
.....
.....

3. input : l'ordinateur écoute

Jusqu'ici, c'est toi qui as collé « Awa » dans le code. Avec `input`, le programme **demande** et attend que tu tapes une réponse.

```
prenom = input("Ton prénom : ")
print("Bonjour", prenom)
```

L'ordinateur affiche `Ton prénom :`, tu tapes, tu valides avec Entrée, puis il dit bonjour.

À RETENIR

`input` rend **toujours un texte** (`str`), même si tu tapes `20`. Pour calculer avec un âge, tu convertis :
`age = int(input("Ton âge : "))`.

4. if : une décision

Un programme qui fait toujours la même chose, ce n'est pas encore très vivant. `if` veut dire : « si c'est vrai, fais ceci ».

```
age = int(input("Ton âge : "))
if age >= 18:
    print("Tu peux voter.")
else:
    print("Pas encore.")
```

Analogie feu : vert → tu avances. Rouge → tu t'arrêtes. Le `if` choisit la branche.

L'indentation, c'est le bloc

En Python, les espaces en début de ligne **comptent**. Ce qui est en retrait sous le `if` appartient au « si ». On aligne avec 4 espaces (la touche Tab, souvent).

COMPARAISON	SENS
<code>==</code>	Est-ce égal ? (question)
<code>=</code>	Range dans la boîte (ordre)
<code>!=</code>	Est-ce différent ?
<code>></code> <code><</code> <code>>=</code> <code><=</code>	Plus grand / plus petit

= N'EST PAS ==

`prenom = "Awa"` range. `prenom == "Awa"` pose la question. Si tu écris `if prenom = "Awa":`, Python refuse.

Un troisième chemin : `elif`

```
reponse = input("Tu as déjà codé ? (oui/non) : ")
if reponse == "oui":
    print("Bienvenue, on accélère un peu.")
elif reponse == "non":
    print("Parfait, c'est le public du soir.")
else:
    print("Réponds par oui ou non.")
```

Les quatre erreurs du soir

Quand Python s'arrête, lis la **dernière ligne** du message. C'est souvent suffisant.

ERREUR	ÇA VEUT DIRE	RÉFLEXE
<code>SyntaxError</code>	La phrase est mal formée : parenthèse, deux-points, guillemet.	Compte les <code>(</code> et les <code>"</code> .
<code>NameError</code>	Python ne connaît pas ce nom.	Variable oubliée, ou texte sans guillemets.
<code>IndentationError</code>	Le retrait sous le <code>if</code> manque ou dépasse.	Aligne avec 4 espaces.
<code>ValueError</code>	<code>int("Awa")</code> : on ne peut pas transformer ça en nombre.	Vérifie ce que tu tapes dans <code>input</code> .

2.1 · 5 MIN · QU'EST-CE QUI S'AFFICHE ?

Si tu tapes `16` puis Entrée :

```
age = int(input("Âge : "))
if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

2.2 · 5 MIN · RÉPARE

```
ville = input("Ville : ")
if ville = "Lomé"
print("On est au Togo.")
```

Deux problèmes au moins. Note-les :

1.
2.

2.3 · 6 MIN · UN ACCUEIL QUI DÉCIDE

Demande le prénom. Si c'est le tien, affiche « C'est moi ! ». Sinon, affiche « Enchanté, » suivi du prénom tapé.

Un script écrit le texte de ta vitrine

Hier, tu as tapé le texte à la main dans `index.html`. Ce soir, Python le compose. Tu le recopieras ensuite dans ta page — même vitrine, autre métier.

FICHIER À CRÉER

Dans le dossier `vitrine` : `bio.py` (pas `.txt`, pas `.html`).

Cahier des charges · non négociable

- ☐ Le fichier s'appelle `bio.py`.
- ☐ Il demande ton prénom avec `input`.
- ☐ Il demande une phrase sur toi (2 ou 3 phrases collées, ou une seule).
- ☐ Il demande 3 envies pour la semaine.
- ☐ Il affiche un texte propre, prêt à coller dans la vitrine.
- ☐ Si le prénom est vide, il dit « Il me faut un prénom. » et s'arrête.

Modèle à adapter (change Awa)

Tape-le, ne le copie pas en silence. Remplace les exemples par tes réponses en live.

```
prenom = input("Ton prénom : ")

if prenom == "":
    print("Il me faut un prénom.")
else:
    phrase = input("Une phrase sur toi : ")
    envie_1 = input("Envie 1 : ")
    envie_2 = input("Envie 2 : ")
    envie_3 = input("Envie 3 : ")

    print()
    print("--- Texte pour ta vitrine ---")
    print(prenom)
    print(phrase)
    print("Cette semaine je veux :")
    print("-", envie_1)
    print("-", envie_2)
    print("-", envie_3)
```

ENSUITE, DANS INDEX.HTML

Copie le texte affiché : le `<h1>` reçoit le prénom, le `<p>` reçoit la phrase, la liste reçoit les 3 envies. Tu n'es pas obligé de tout recoller ce soir : le script qui marche suffit pour valider. Recoller, c'est le bonus.

Les cinq blocages classiques

SYMPTÔME	RÉFLEXE
Le fichier s'appelle <code>bio.py.txt</code>	Affiche les extensions dans l'explorateur Windows.
« python n'est pas reconnu »	Passe sur online-python.com . On installe plus tard.
Rien ne s'affiche, le curseur clignote	Le programme attend <code>input</code> . Tape une réponse, Entrée.
<code>IndentationError</code>	Tout ce qui est sous <code>else:</code> est en retrait, aligné.
Les accents cassent dans le Bloc-notes	Enregistre en UTF-8, ou code en ligne.

Comment lancer le fichier (si Python est installé)

1. Ouvre un terminal dans le dossier `vitrine` .
2. Tape `python bio.py` (parfois `py bio.py` sous Windows).
3. Réponds aux questions, Entrée après chaque ligne.

MINI-EXO DE FIN DE SÉANCE

Dépose `bio.py` sur MyHub, espace étudiant, devoir du jour 2. Tu peux coller le code si le fichier refuse de partir.

Inscription : prénom, nom, téléphone. Connexion : prénom + téléphone.

Glossaire du jour

MOT	EN UNE PHRASE
Programme	Une liste d'instructions exécutées dans l'ordre.
Python	Le langage de ces instructions, ce soir.
Interpréteur	Le programme qui lit et exécute le fichier <code>.py</code> .
<code>print</code>	Affiche un message à l'écran.
Variable	Une boîte nommée qui range une valeur.
<code>=</code>	Range une valeur dans une variable.
<code>str</code>	Un texte, entre guillemets.
<code>int</code>	Un nombre entier, sans guillemets.
<code>input</code>	Demande une réponse ; rend toujours un texte.
<code>if / else</code>	Choisit une branche selon une condition.
<code>==</code>	Compare : est-ce égal ?
Indentation	Le retrait qui marque le bloc sous un <code>if</code> .
Erreur	Python s'arrête et explique, souvent à la dernière ligne.

Checklist avant d'envoyer ton fichier

- ☐ Le fichier s'appelle `bio.py`.
- ☐ Il y a au moins un `input` pour le prénom.
- ☐ Il y a un `if` (prénom vide, ou un autre choix clair).
- ☐ Tu l'as lancé une fois : le texte s'affiche.
- ☐ Les 3 envies apparaissent à l'écran.
- ☐ Tu déposes sur MyHub, pas sur WhatsApp.

POUR LE JOUR 3 · DATA

Ne jette ni `vitrine` ni `bio.py`. Demain, un invité collera 3 chiffres sur ta page. Même heure, même visio, ordinateur encore obligatoire.